The example above uses an anonymous function, but it is perfectly fine to use named functions or variables that reference a function. The other callback-based regex replacement function is `preg_replace_callback_array()`, which works a bit differently: an array is passed, which uses the keys as regex patterns, and the elements are the callback functions.

```
$callbacks = [
    '/\d+/' => function (array $matches): string {
        return number_format($matches[0], thousands_separator: ',');
    },
    '/\w+/' => function (array $matches): string {
        return strtoupper($matches[0]);
    },
];
print preg_replace_callback_array($callbacks, 'PoWeR LeVeL: 9001');
```

```
Output:
POWER LEVEL: 9,001
```

preg_split()

Like `explode()`, `preg_split()` takes a single input string and divides it into an array of sub-strings according to a pattern. If the pattern is very simple, it is recommended to use `explode()` for better performance; otherwise `preg_split()` can save you from needing a lot of tortuous string-handling logic:

```
$text = 'We just, want.the - words...from-this??string.';
print_r(preg_split("/[^a-zA-Z]+/", $text));
```

```
Output:
Array
(
    [0] => We
    [1] => just
    [2] => want
    [3] => the
    ...
    [8] =>
)
```